

Package ‘cncleanr’

September 30, 2026

Title Parse Compact Numbers in Chinese Data

Version 0.2.5

Description Parses compact numeric values, qualified quantities, and ranges commonly found in Chinese tables and spreadsheets. It handles Chinese magnitude suffixes, currencies, percentages, full-width characters, financial negatives, and configurable missing-value markers while reporting values that cannot be parsed safely.

License MIT + file LICENSE

URL <https://github.com/Jorungandr/cncleanr>

BugReports <https://github.com/Jorungandr/cncleanr/issues>

Encoding UTF-8

Language en-US

Suggests spelling, testthat (>= 3.0.0)

Config/testthat/edition 3

Config/roxygen2/version 8.1.0

NeedsCompilation no

Author Haotian Liu [aut, cre, cph]

Maintainer Haotian Liu <wyvern06@163.com>

Repository CRAN

Date/Publication 2026-09-30 09:30:02 UTC

Contents

cn_problems	2
parse_cn_number	2
parse_cn_quantity	3
parse_cn_range	4
Index	6

cn_problems

Extract Problems From a cncleanr Result

Description

Returns parsing failures attached to an object produced by a cncleanr parser. Successful results return an empty data frame with the same stable schema.

Usage

```
cn_problems(x)
```

Arguments

x An object returned by a cncleanr parsing function.

Value

A data frame with columns index, value, and reason.

Examples

```
result <- suppressWarnings(parse_cn_number(c("bad", "2\u4e07")))
cn_problems(result)
```

parse_cn_number

Parse Compact Numbers Used in Chinese Data

Description

Converts character values such as "1.25\u4e07", "\uffe53\u4ebf", "1.2e5", and "12.5%" to doubles. Parsing is deliberately conservative: every non-missing value must match the supported syntax in full. Spaces between digits are accepted only when they form valid three-digit grouping.

Usage

```
parse_cn_number(
  x,
  na = c(
    "", "NA", "N/A", "\u6682\u65e0", "\u672a\u516c\u5e03",
    "\u2014", "\u2013", "...", "\u2026"
  ),
  strict = FALSE
)
```

Arguments

<code>x</code>	A character, factor, or numeric vector. Numeric input is converted to double and otherwise passed through unchanged, including Inf, -Inf, and NaN.
<code>na</code>	A character vector containing values that should be interpreted as missing. Matching happens after whitespace and full-width normalization.
<code>strict</code>	A single logical value. If FALSE, invalid values become NA_real_, a warning is emitted, and a data frame is stored in the problems attribute. If TRUE, invalid values cause an error.

Details

Finite-range validation applies to parsed character and factor input. Existing numeric Inf, -Inf, and NaN values are preserved.

Value

A double vector with the same length and names as `x`. When invalid values occur in non-strict mode, the result has a problems attribute with columns index, value, and reason.

Examples

```
parse_cn_number(c("1.25\u4e07", "3\u4ebf\u5143", "12.5%", "\u6682\u65e0"))
parse_cn_number(c("\uff11\uff12\uff0e\uff15\uff05", "(2.5\u4e07)"))
```

parse_cn_quantity	<i>Parse Qualified Quantities Used in Chinese Data</i>
-------------------	--

Description

Parses exact values and values qualified by language or symbols, such as "\u7ea3\u4e07", "\u5927\u4e8e\u4ebf", "\u4e0d\u5c0f\u4e8e\u4e07", "10\u4e07+", "50\u4f59", or "50\u4f59\u4e07\u5143". The qualifier is retained instead of silently discarded.

Usage

```
parse_cn_quantity(
  x,
  na = c(
    "", "NA", "N/A", "\u6682\u65e0", "\u672a\u516c\u5e03",
    "\u2014", "\u2013", "-", "...", "\u2026"
  ),
  strict = FALSE
)
```

Arguments

x	A character, factor, or numeric vector. Numeric input is converted to double and otherwise passed through unchanged, including Inf, -Inf, and NaN.
na	A character vector containing values that should be interpreted as missing. Matching happens after whitespace and full-width normalization.
strict	A single logical value. If FALSE, invalid values become NA_real_, a warning is emitted, and a data frame is stored in the problems attribute. If TRUE, invalid values cause an error.

Value

A data frame with double column value and character column qualifier. Qualifiers are exact, approx, greater_than, at_least, less_than, and at_most. Numeric finite and infinite values have the qualifier exact; NA and NaN have a missing qualifier. Numeric values, including Inf, -Inf, and NaN, are preserved in value.

Examples

```
parse_cn_quantity(c(
  "\u7ea63\u4e07", "\u5927\u4e8e2\u4ebf", "10\u4e07+", "50\u4f59", "50\u4f59\u4e07\u5143"
))
```

parse_cn_range	<i>Parse Numeric Ranges Used in Chinese Data</i>
----------------	--

Description

Converts closed ranges such as "3\u4e07-5\u4e07" and open bounds such as "10\u4e07\u5143\u4ee5\u4e0a" into explicit lower and upper bounds. A shared suffix in "3-5\u4e07" is applied to both endpoints. Signs used by negative numbers and scientific notation are distinguished from range separators.

Usage

```
parse_cn_range(
  x,
  na = c(
    "", "NA", "N/A", "\u6682\u65e0", "\u672a\u516c\u5e03",
    "\u2014", "\u2013", "-", "...", "\u2026"
  ),
  strict = FALSE
)
```

Arguments

x	A character, factor, or numeric vector. Numeric input is converted to double and otherwise passed through unchanged, including Inf, -Inf, and NaN.
na	A character vector containing values that should be interpreted as missing. Matching happens after whitespace and full-width normalization.
strict	A single logical value. If FALSE, invalid values become NA_real_, a warning is emitted, and a data frame is stored in the problems attribute. If TRUE, invalid values cause an error.

Value

A data frame with columns lower, upper, lower_inclusive, and upper_inclusive. Numeric input becomes an inclusive point range, including numeric Inf and -Inf; NA and NaN have missing inclusivity. Infinite bounds created from textual inequalities represent open-ended bounds and are distinct from numeric point inputs.

Examples

```
parse_cn_range(c("3\u4e07-5\u4e07", "3-5\u4e07", "10\u4e07\u5143\u4ee5\u4e0a"))
```

Index

`cn_problems`, [2](#)

`parse_cn_number`, [2](#)

`parse_cn_quantity`, [3](#)

`parse_cn_range`, [4](#)